# FAPS-3D: Feedback-directed Adaptive Page Management Scheme for 3D-Stacked DRAM

Muhammad M. Rafique
Department of Electrical and Computer Engineering
University of Illinois at Chicago
Chicago, Illinois, USA
mrafiq2@uic.edu

Zhichun Zhu
Department of Electrical and Computer Engineering
University of Illinois at Chicago
Chicago, Illinois, USA
zzhu@uic.edu

## ABSTRACT

Modern multiprocessor systems with multiple applications running concurrently lead to random memory access patterns during different phases of execution. Traditional static memory page policies used to exploit locality and minimize the access latency are not optimal under these varying memory access patterns. It requires a dynamic page management scheme to cope with these random memory access sequences to minimize memory access latency while maximizing available bandwidth utilization. The emergence of 3D-stacked DRAM provides several architectural advantages such as hundreds of memory banks to boost memory-level parallelism and availability of logic area to integrate custom logic in DRAM module. This paper introduces an adaptive page management scheme for 3D-stacked DRAM based main memory systems, which aims to minimize memory access latency by dynamically adjusting page modes based on the changing memory access patterns across different banks. The simulation results indicate that our scheme improves performance by 15.2% over the base *close-page* mode and by 4.0% over an existing *access-based* page management policy on average when running multi-programming SPEC workloads of varying memory intensities. It also outperforms a state-of-the-art *time-based* page management policy by 3.3%. Our scheme is implemented in the logic base of 3D-stacked DRAM module and has negligible area and energy overheads.

## CCS CONCEPTS

• **Computer systems organization** → Processors and memory architectures; • **Hardware** → Semiconductor memory

## KEYWORDS

Memory Systems, 3D-Stacked DRAM, Page Mode.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

*MEMSYS '19*, September 30-October 3, 2019, Washington, DC, USA
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-7206-0/19/09...$15.00
https://doi.org/10.1145/3357526.3357540

## 1. INTRODUCTION

High latency and limited bandwidth of existing main memory system have been a performance bottleneck for modern computer systems [1]. This problem is exacerbated by the fact that processing power has been increased tremendously by the introduction of multiple processing cores on a single chip. The emerging data-intensive applications running concurrently on the multi-core processor generates interleaved, random memory requests. The statically implemented page policy, either *open-page* or *close-page,* yields sub-optimal performance in serving these random memory requests.

The emergence of 3D-stacked DRAM addresses the bandwidth limitations by incorporating extensive design improvements over traditional DDRx-based main memory systems [2], [26], [36]. The 3D stacking of DRAM dies over a logic die provides excellent concurrency as hundreds of memory banks spread across multiple *vaults* are accessible through thousands of very high-speed *Through-Silicon Vias* (TSVs) [8]. Each vault is managed by a vault-controller residing in its logic base and is operationally and functionally independent, providing the freedom to implement its own page and scheduling policies that suit best to its share of memory requests. Figure 1 shows the overview of 3D-stacked DRAM module.

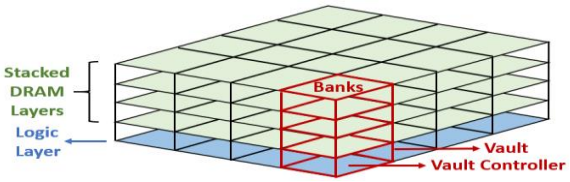



**Figure 1. Overview of 3D-Stacked DRAM**

In main memory system, to maximize the utilization of available bandwidth and minimize the memory access latency, memory transactions are pipelined by providing multiple banks where each bank can be accessed independently. Data is mapped across different channels, ranks and banks to overlap the transaction time of multiple memory requests. Based on the locality found within the applications, different page policies yield different memory access latencies. For applications that exhibit higher spatial locality, *open-page* mode is implemented where the recently accessed row is kept open in row-buffer, so that future

requests to the same row only need to perform column accesses [5]. On the other hand, applications exhibiting lower spatial locality benefit more from *close-page* mode where a row-buffer is precharged after the current access provided there is no pending request targeting at the row. Both *open* and *close-page* policies have their pros and cons. The actual performance impact depends on the number of application threads running concurrently and their locality.

In order to make main memory system robust to randomly generated memory access requests in many-core environment, optimizations need to be done in memory controller including page mode, scheduling policy and management of refresh operations. Dynamic page policies have been studied to reduce bank conflicts and overall memory access latency. One class of the dynamic page policies is called *access-based policies* (ABP) [13], [25], [29], where *row hits* and *row conflicts* are monitored at different granularities in DRAM and future page policy is predicted based on the past row hit/conflict behaviors. Another class of dynamic page policies is called *time-based policies* (TBP) [30], [34], [35], where a page is kept open for certain time period and then precharged. The threshold of keeping page open is dynamically adjusted based on various prediction algorithms. These schemes have been designed for traditional DDRx-based main memory systems and may not be optimized to take advantage of 3D-stacked DRAM structures such as the flexibility to manage each vault according to its own memory access pattern and the availability of logic area to implement custom logic in order to minimize the access latency and energy consumption.

Changing application behavior results in varying row-buffer hit-rate throughout the program execution and should be considered while designing efficient memory management policies. As we will see shortly, even within the same application, there are different phases of program execution. In some phases, data is accessed in a streamline fashion and take advantage of the locality throughout the memory hierarchy; while in the other phases, data access patterns are irregular, which result in frequent cache misses and additional references to the main memory. In either case, the memory access behavior is not known ahead of time and some mechanism is required to learn the application behavior during run-time to better manage the memory requests.

In this paper, we propose *FAPS-3D*, **F**eedback-directed **A**daptive **P**age management **S**cheme for **3D**-stacked DRAM, a dynamic page management scheme implemented by a *policy engine* that resides in the logic base of each vault. This engine profiles the access patterns of each bank within the vault and guides the vault controller to adjust the page mode accordingly. The *policy engine* works in a feedback fashion and adjusts its guidance to the vault controller based on the continuously varying row-buffer hit-rate and overall bank access latency. This mechanism totally operates in the logic base of 3D-stacked DRAM module and requires no major modification in the main memory controller. It gathers local information to guide the vault controller in efficient management of the memory requests.

The major contributions of this work are as follows:

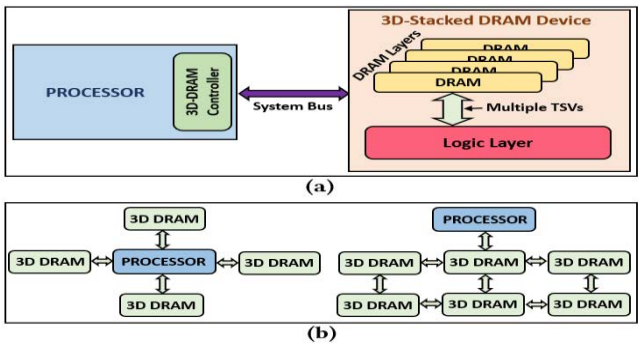



**Figure 2. (a) 3D-Stacked DRAM Device Architecture, (b) Network of 3D-Stacked DRAM Devices**

- We analyze the possibilities of exploiting key structural advantages of 3D-stacked DRAM such as huge number of memory banks, availability of the logic area and independent nature of vault operations to enhance the capabilities of vault controller in order to serve memory requests more efficiently.
- We propose an efficient vault management scheme that profiles the access behavior of banks within a vault and dynamically adjusts the page policy to reduce overall memory access latency.
- We demonstrate that in 3D-stacked DRAM based main memory system, adjusting the page mode by keeping track of memory access pattern at the bank level provides performance comparable to the finer-grained page policies that keep track of access pattern at DRAM page level, with considerably less design complexity and area overhead.

Simulation results indicate that our proposed scheme considerably reduces the main memory access latency and improves overall system performance. We evaluated our scheme using eight-core multiprogramming SPEC CPU2006 workload sets of varying memory intensity. Compared to the default *close-page* mode, our scheme achieves performance gain of 15.2% on average. Compared to an existing *access-based* row-buffer management policy, our scheme achieves an average performance improvement of 4.0% while reducing the area overhead by 98x. Our scheme also outperforms a state-of-the-art *time-based* page management scheme by 3.3% on average.

The rest of the paper is organized as follows: Section 2 describes 3D-stacked DRAM architecture and motivation behind our work. Section 3 explains our proposed adaptive page management scheme in detail. Detailed simulation setup and all the schemes used in our evaluation are presented in section 4. Simulation results and analyses are presented in section 5. Section 6 discusses some of the related work and conclusions are drawn in section 7.

## 2. BACKGROUND AND MOTIVATION

In this section, we briefly review 3D-stacked DRAM architecture, its advantages over traditional DDRx-based main memory system and how these architectural advantages can be exploited for better management of memory access requests. We will also analyze varying application's run-time behavior, which is the motivation behind our work.

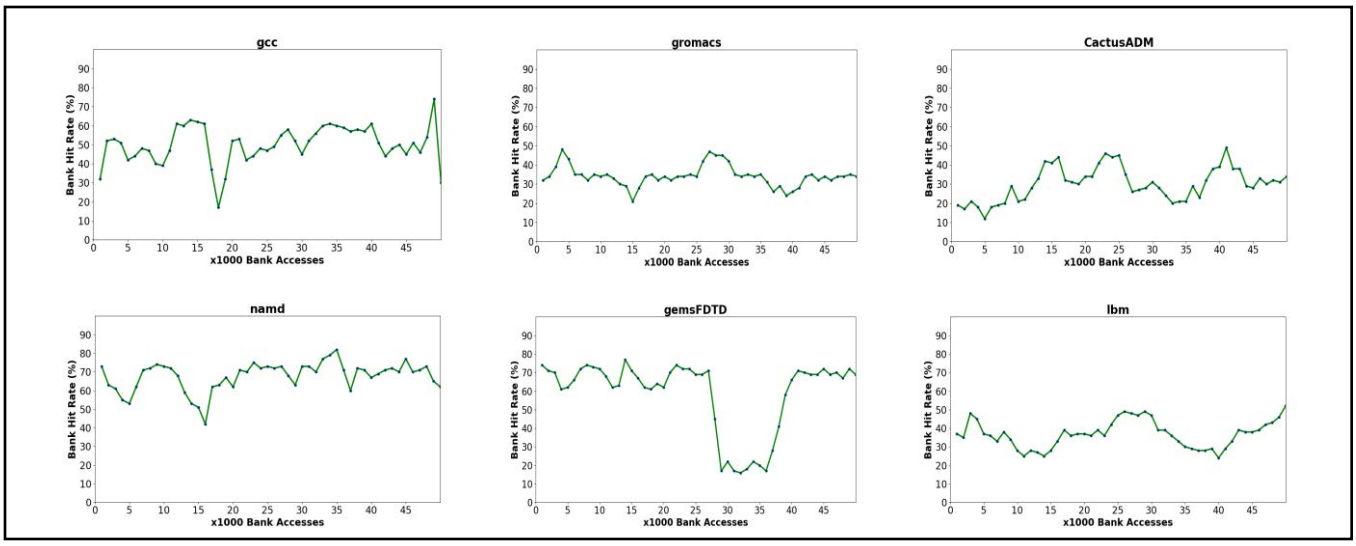



**Figure 3. Row-buffer Hit-Rate of Various SPEC CPU2006 Benchmarks**

## 2.1 3D-Stacked DRAM Architecture

Figure 2(a) shows basic architecture of 3D-stacked DRAM device. Internally, it consists of multiple DRAM layers that are stacked on top of a single logic layer. Each DRAM layer is partitioned into multiple segments and re-architected in such a way that multiple DRAM dies in different layers are connected vertically by very high speed *Through Silicon Vias* (TSVs) to form a column like structure called *vault*. Each vault is operationally and functionally independent to each other and can implement memory management policies to reduce overall access latency and improve throughput. Hundreds of DRAM banks are spread across multiple vaults. This provides excellent memory-level parallelism to carry out multiple independent memory transactions simultaneously, which results in shallow read and write queues and improved memory response time, making it ideal for high-performance and bandwidth-sensitive systems like servers. Examples of such devices include Hybrid Memory Cube (HMC) [2], [11] and High Bandwidth Memory (HBM) [36]. The capacity of 3D-stacked DRAM based main memory system can be increased by connecting multiple modules directly to host CPU or through the network of other modules as shown in Figure 2(b).

## 2.2 Diversity in Application Behavior

The emergence of modern data-intensive applications accelerated the demand of revolutionary improvements in the memory hierarchy design from the underlying base technology all the way up to architectural level. Multiple applications running concurrently on processing cores may impose different memory bandwidth requirements, which ignite the competition on the shared memory resources. Two types of applications are extensively dependent on main memory references. *First*, applications having data set so large that cannot be fit into caches without compromising the demand from other applications running concurrently. *Second*, applications that exhibit very little *spatial* and *temporal* locality, which results in frequent data movement between caches and main memory, incurring additional latency and energy overhead.

The diversity in application behavior can be categorized as follows:

*2.2.1 Inter-Application Diversity:* Multiple applications running concurrently can have different memory access behavior. Previous studies [7], [16], [22] found that different SPEC CPU2006 benchmarks running on single-core processors yield different cache miss rates and row-buffer hit-rates. This diversity in application behavior imposes varying requirement on memory bandwidth and should be managed in a way to ensure minimum response latency for each application thread without causing extensive delays for the others.

*2.2.2 Intra-Application Diversity:* Even within a single application running on a single processing core, the memory bandwidth requirement is not uniform throughout the duration of program execution. Figure 3 shows such behavior of some SPEC CPU2006 benchmarks. These graphs are obtained by running each benchmark on a single-core system and showing the row-buffer hit-rate for a total of 50K accesses, while taking average statistics per 1K accesses. The applications *gromacs*, *cactusADM* and *lbm* show row-buffer hit-rate variation of 20-50%, 12-49% and 30-48%, respectively, while *gcc*, *namd* and *gemsFDTD* have noticeable variation of 16-72%, 40-81% and 17-79%, respectively. This implies that a statically implemented page mode is not optimal during the whole duration of an application's execution.

## 2.3 Motivation

*2.3.1 Limitation of existing main memory systems:* The modern many-core systems running multiple data-intensive applications concurrently require huge bandwidth, which is not provided by existing DDRx-based main memory system. Limited numbers of memory channels along with cumulative energy consumption are also limiting factors in scaling up the bandwidth provided by current memory systems. Widely varying application behavior also imposes some serious challenges on memory controller in implementing adaptive memory management policies to cope with the application's continuously changing run-time behavior.

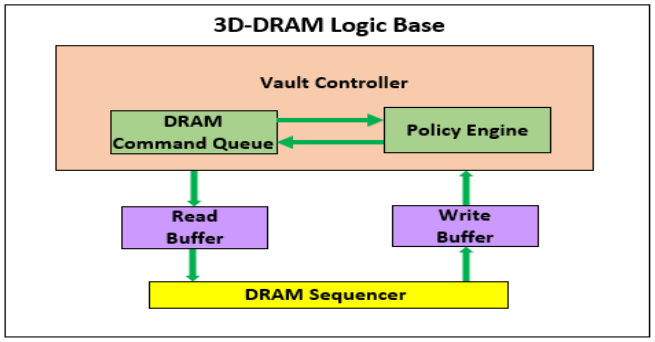



**Figure 4. Policy engine in 3D-DRAM Logic Base**

Adding these capabilities would incur additional design complexities in a single, centralized memory controller in terms of area and energy overhead, which is a vital design consideration for modern microprocessor design. These limitations require radical changes in main memory design and management to improve overall system performance.

*2.3.2 Advantages of 3D-Stacked DRAM Architecture:* The challenges mentioned above in current memory systems have been addressed to some extent by the 3D-stacked DRAM in following ways:

- Huge number of available banks spread across multiple vaults provide high concurrency and substantial memory-level parallelism to carry out multiple transactions simultaneously.
- Availability of logic base provides the platform to integrate custom logic that can enhance the operational and functional capabilities of vault controller in order to better manage the memory requests.
- Lower level memory operations are taken care by the vault controller, taking this burden off the main memory controller that resides on the host processor. This makes these modules compatible for a wide variety of platforms and applications.
- Availability of very high-speed memory bus that connects processor with the 3D-stacked DRAM module provides exceptional external bandwidth for seamless communication between processor and the memory device.

Considering the challenges associated with existing main memory systems, the diversity in the memory access behavior of different applications and the revolutionary 3D-stacked DRAM architecture, we now introduce our adaptive page management scheme that extensively utilizes key 3D-stacked DRAM features to efficiently serve the memory requests under these varying load conditions.

## 3. ADAPTIVE PAGE MANAGEMENT SCHEME

The 3D-stacked DRAM receives memory access requests from processor that travel over very high-speed memory bus and are distributed across multiple vaults by a crossbar switch residing in the logic base. Each vault controller has the freedom to implement its own vault management policies and determines its own timing requirement that suits best to serve its share of memory requests [11]. We used this key feature to develop our vault management scheme in a dedicated *policy engine* (Figure 4)

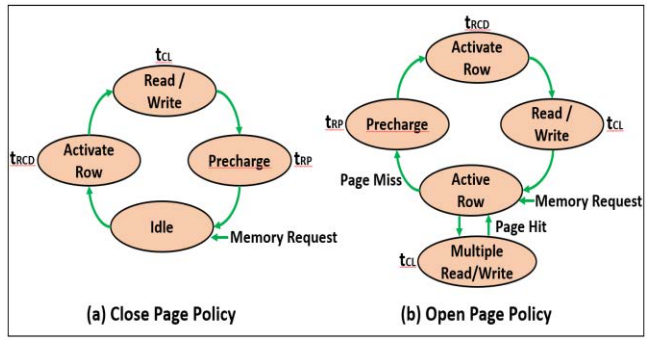



**Figure 5. State diagram for Open and Close page policies**

that resides in the vault controller. The *policy engine* works in coordination with the DRAM command queue and these operations are fully automated and transparent to the memory controller residing on the processor chip. This makes the implementation of our vault management scheme much simpler and incurs very low area overhead.

### 3.1 Policy Engine Operations

The *policy engine* resides in the vault controller and closely monitors the access pattern for all banks within the vault. It profiles the bank's *hit-rates* and directs the issue of lower level DRAM commands like activate/precharge and refresh operations to ensure lower latency while respecting the timing constraints associated with these operations [3]. The policy engine works in an adaptive fashion and updates its guidance to DRAM command queue based on the target address of incoming requests, current status of the banks and read/write queues, and availability of *TSVs* within the vault to carry out actual memory bank transaction. Here are the details of operations performed by the policy engine:

*3.1.1 Threshold Selection:* The most important decision made by the *policy engine* is to manage all the banks in the vault based on the row-buffer hit-rate as criterion to select the *open-page* or *close-page* mode. To perform this task, a cutoff in row-buffer hit-rate needs to be carefully selected to minimize the overall bank access latency for both *read* and *write* operations.

**Table 1: Latencies associated with different page modes**

| | | |
|---|---|---|
| Open Page | Page Hit | $t_{CL}$ |
| | Page Conflict | $t_{RP} + t_{RCD} + t_{CL}$ |
| Close Page | Page Miss | $t_{RCD} + t_{CL}$ |

In *open-page* mode, a row cached in the row-buffer is not *precharged* immediately after the current access in anticipation that the next access will be a hit in the same row. It only performs a precharge operation when there are no more hits to the open row and there is a bank conflict, which means upcoming request goes to a different row within the same bank. In this policy, *precharge* latency is on the critical path and causes additional delays in case of bank conflicts. The *close-page* mode, on the other hand, precharges the row after the access, provided currently no other requests go to the same row. It frees the row-buffer for the next request, taking the *precharge* latency off the critical path. Figure 5 shows the state diagram for *open* and close

**Algorithm I**: For banks currently having *open-page* mode

**Definition:**

N: number of banks in *vault* with *open-page* mode currently
$bank_{ip}$: *Bank policy* of bank *i* for next epoch
$bankHitAvg_i$: Average bank hit-rate per epoch
$b_a$: total bank accesses, $b_h$: total bank hits
$bank_{cs}$: Current bank state
$bank_{ns}$: Next bank state
Low threshold ($th_l$) = 25%, threshold (th) = 50%

```
for each epoch do
  for i = 1 to N do
    bankHitAvg_i(b_a, b_h)  // calculate average bank hit-rate
    if bankHitAvg_i(b_a, b_h) < th_l then
      bank_ns ← (00)_2
      bank_ip ← closePage // bank i will follow Algorithm II in next epoch
    else if bankHitAvg_i(b_a, b_h) < th then
      bank_ns ← bank_cs - 1 // saturated decrement
    else // bankHitAvg_i(b_a, b_h) ≥ th
      bank_ns ← bank_cs + 1 // saturated increment
    end if
    if bank_ns ≤ (01)_2 then
      bank_ip ← closePage // bank i will follow Algorithm II in next epoch
    else // (10)_2 or (11)_2
      bank_ip ← openPage // bank i will follow Algorithm I in next epoch
    end if
  end for
end for
```

**Algorithm II**: For banks currently having *close-page* mode

**Definition:**

N: number of banks in *vault* with *close-page* mode currently
$bank_{ip}$: *Bank policy* of bank *i* for next epoch
$bankHitAvg_i$: Average possible bank hit-rate per epoch
$b_a$: total bank accesses, $b_{ph}$: total *possible* bank hits
$bank_{cs}$: Current bank state
$bank_{ns}$: Next bank state
High threshold ($th_h$) = 75%, threshold (th) = 50%

```
for each epoch do
  for i = 1 to N do
    bankHitAvg_i(b_a, b_ph)  // calculate possible average bank hit-rate
    if bankHitAvg_i(b_a, b_ph) ≥ th_h then
      bank_ns ← (11)_2
      bank_ip ← openPage // bank i will follow Algorithm I in next epoch
    else if th ≤ bankHitAvg_i(b_a, b_ph) then
      bank_ns ← bank_cs + 1 // saturated increment
    else // bankHitAvg_i(b_a, b_ph) < th
      bank_ns ← bank_cs - 1 // saturated decrement
    end if
    if bank_ns ≥ (10)_2 then
      bank_ip ← openPage // bank i will follow Algorithm I in next epoch
    else // (01)_2 or (00)_2
      bank_ip ← closePage // bank i will follow Algorithm II in next epoch
    end if
  end for
end for
```

*page* policies. The latencies associated with these page policies are given in Table 1. Here $t_{RCD}$ represents latency to *activate* the row, $t_{CL}$ represents latency to select a certain column from the activated row and $t_{RP}$ represents the latency to *precharge* currently open row.

For *open-page* mode, each access is either a row-buffer hit or row-buffer conflict, while for the *close-page* mode, nearly all accesses result in row-buffer misses. For certain banks, if the row-buffer hit-rate is very low, the precharge latency considerably increases the overall bank access latency (see Table 1). For such banks, *close-page* mode is more suitable as it will save the latency associated with precharge operation. Most of the current DRAM devices have comparable delays associated with activate, precharge and column access commands [3], which implies the latency of conflicts is three-times higher as that of hits. Therefore, if the row-buffer hits are less than half of the total bank accesses, the extra delay associated with the conflicts offsets the time saved by the row-buffer hits and bank accesses will suffer from additional delay than they would have if *close-page* mode was implemented. Based on these facts, our scheme dynamically profiles banks as candidates of *open/close* page policies.

*3.1.2 Dynamic Bank Profiling:* Based on the threshold selection criterion described above, the policy engine profiles the numbers of row-buffer hits and accesses for each bank in the vault. Initially, the scheme applies *open-page* mode and scheduling policy that gives preference to row-hits over all other accesses [5]. Our *policy engine* then computes the row-buffer hit-rate of each bank per 1000 accesses (epoch length) and classifies the banks with hit-rate greater than 75% as *HHR* (high-hit-rate) banks and banks with hit-rate less than 25% as *LHR* (low-hit-rate) banks. The banks with hit-rate in range of 25% to 75% are classified as *MHR* (medium-hit-rate) banks. These thresholds are empirically selected and are justified later by simulation results. To classify banks in each group, the *policy engine* maintains a 2-bit saturating counter per bank as a *finite state machine* (FSM). The states labelled as $(10)_2$ and $(11)_2$ would recommend *open-page* mode and states labelled as $(01)_2$ and $(00)_2$ would recommend *close-page* mode for next epoch.

*3.1.3 Policy for open-page banks:* For banks that currently have an *open-page* mode, the policy engine simply calculates the row-buffer hit-rate for each bank by dividing the total number of page hits in each bank by the total number of accesses. If a bank has very low hit-rate, i.e. 25% or less, the bank belongs to *LHR* group; and the FSM is transitioned directly to state $(00)_2$ irrespective of the current state and the *close-page* mode is implemented for next epoch. Otherwise, the counter value is decremented by one if hit-rate is between 25% and 50% and is incremented by one if hit-rate is greater than 50%. At the end of epoch, based on the current FSM state, all banks are individually classified to be candidates of *open* or *close-page* mode and corresponding page mode is implemented for the next epoch. The details of this policy are given in Algorithm I.

*3.1.4 Policy for close-page banks:* The banks that currently have *close-page* mode implemented, are re-evaluated based on different criteria since *close-page* mode apparently have no bank hits. To profile the access behavior for each bank, an SRAM-based "*hit-register*" is used to keep track of the number of "*potential hits*" if the next access goes to the same page as the previous one. At the end of epoch, "*potential bank hit-rate*" (PBHR) is calculated by dividing the number of *potential hits* by total number of accesses. If *PBHR* is greater than 75%, the bank belongs to *HHR* group and *FSM* is transitioned directly to state $(11)_2$ irrespective of current state, and *open-page mode* is implemented for the next epoch. Otherwise, the counter value is

**Table 2: Baseline Simulation Configuration**

| Parameter | Configurations |
|---|---|
| Processor | 8 out-of-order cores @ 3.0 GHz, 8-issue superscalar, x86-64 ISA, 192 entry ROB. |
| Caches | L1 (I/D): 32KB (P), 4 way, 64B line, $t_{hit}$ = 4 cycles, 16 MSHRs<br>L2: 256KB (P), 8 way, 64B line, $t_{hit}$ = 10 cycles, 16 MSHRs<br>L3: 16MB (S), 16 way, 64B line, $t_{hit}$ = 20 cycles, 64 MSHRs<br>Multi-banked, Inclusive, MOESI cache coherence protocol |
| 3D-stacked Memory | 8 GB, 32 vaults, 8 DRAM layers, 16 banks/vault, 64 TSVs/vault, 1.25 Gb/s TSV signaling rate. |
| DRAM Specs. | DDR3-1600, Queue Size (R/W) = 32/64<br>1KB row-buffer size, 16384 rows/bank<br>$t_{rcd}$ = 11 cycles, $t_{rp}$ = 11 cycles, $t_{cl}$ = 11 cycles. [3] |
| Vault Controller | Memory Scheduling: First-Come First-Serve<br>Address Mapping: Row:Rank:Bank:Column:Vault<br>Page mode: Close Page |

incremented by one if the *PBHR* is between 50% and 75% and is decremented by one if the *PBHR* is less than 50%. Eventually, based on the current *FSM* state, all banks are individually marked to implement *close* or *open-page* mode for next epoch. Algorithm II shows the details of this policy.

## 3.2 Overhead

Our proposed adaptive page management scheme introduces some overhead. There is a *policy engine* residing in the logic base containing *FSM* and an *SRAM* based *hit-register* to hold information about the most recently accessed DRAM-pages. There are 16 banks in each vault and the *hit register* holds 16 entries (one per bank), with each entry being 52 bits to keep bank ID, total bank access count, last accessed page id, currently accessed page id and total bank hit count. It gives a storage overhead of 16 * 52 = 832 bits or 104 bytes. Across all 32 vaults, our scheme introduces storage overhead of 3.25KB, which is considerably less (98x) than 320KB required to implement the scheme in a previous study [13]. The total number of counters used are (Vaults × Ranks × Banks) × 2, which turns out to be 1024 in our experiment. Area and energy overhead of implementing SRAM-based storage using 32nm technology, is estimated using CACTI 6.0 [36] and turned out to be 0.08mm$^2$ and 0.023 nJ/access respectively. There is an expensive division operation performed to calculate bank hit rate every epoch. Since division operation is performed every few thousand cycles, the energy overhead associated with this operation is trivial and can be ignored. All the simple logic needed in implementing our algorithm can be easily integrated into the logic base of 3D-stacked DRAM.

# 4. EXPERIMENTAL METHODOLOGY

## 4.1 Simulation Configuration

We evaluate our feedback-directed adaptive page management scheme using cycle accurate x86-64 simulator gem5 [14]. The gem5 simulator is extended to support five different page policies including close-page, open-page, prediction-based DRAM row-buffer management [13], Intel-adaptive HAPPY [28], [35] and our scheme FAPS-3D. The details of our baseline simulation configuration are shown in Table 2. Our simplified and accurate 3D-DRAM model is based on official specification released by *Micron*® [2] and previously published literatures in [4], [7], [18], [33]. The energy consumption of 3D-stacked DRAM is estimated using model presented in [32], [33]. The detailed memory controller operations are based on the model presented in [10] whereas electrical and timing parameters are obtained from [3]. The baseline scheme implements *close-page* mode, scheduling policy is chosen to be first-come first-serve (FCFS) and address mapping policy is chosen in such a way that lowest order bits are assigned to vault selection to maximize the parallelism (at the cost of power). For *open-page* mode, scheduling policy is chosen to be first-ready first-come first serve (FR-FCFS) [5] and address mapping is chosen in such a way that lowest order bits denote the column address to ensure that sequential cache lines occupy the same row to maximize page hits.

**Table 3: SPEC CPU2006 Benchmark Sets**

| ID | Multiprogrammed Workload Sets |
|---|---|
| HM1 | bwaves, soplex, lbm, libquantum, lbm, omnetpp, sphinx, libquantum |
| HM2 | lbm, gcc, bwaves, libquantum, soplex, sphinx, gems, soplex |
| HM3 | gcc, libquantum, milc, mcf, leslie3d, lbm, sphinx, soplex |
| HM4 | gcc, gems, leslie3d, lbm, omnetpp, sphinx, leslie3d, gems |
| LM1 | astar, bzip2, cactus, gromacs, bzip2, hmmer, gromacs, astar |
| LM2 | h264ref, bzip2, astar, cactus, wrf, astar, h264ref, gromacs |
| LM3 | cactus, astar, hmmer, zeusmp, bzip2, gromacs, wrf, h264ref |
| LM4 | bzip2, cactus, zeusmp, gromacs, astar, wrf, hmmer, cactus |
| MX1 | gcc, mcf, lbm, sphinx, astar, cactus, bzip2, gromacs |
| MX2 | lbm, bwaves, soplex, libquantum, h264ref, astar, wrf, bzip2 |
| MX3 | gcc, leslie3d, lbm, gems, hmmer, wrf, bzip2, gromacs |
| MX4 | bwaves, leslie3d, soplex, mcf, zeusmp, astar, cactus, h264ref |

## 4.2 Workloads

We evaluate our scheme using SPEC CPU2006 Benchmarks [20], [21]. We used 12 different sets of multiprogramming workloads and each set is categorized based on *misses-per-kilo instructions* (MPKI) values, similar to [7]. These MPKI values are determined by running each SPEC CPU2006 benchmark on single-core system with last level cache size of 512KB. Each set contains a mix of eight different SPEC CPU2006 benchmarks, hence giving a wide range of memory intensive workloads. Based on the range of MPKI values, the workload sets are categorized as follows:

High Memory Intensive (HM): $MPKI_{LLC} > 20$

Low Memory Intensive (LM): $1 \leq MPKI_{LLC} \leq 20$

Mixed Memory Intensive (MX): 4 (HM) + 4 (LM)

The details of each workload set are shown in Table 3. For each SPEC CPU2006 benchmark, "*reference*" data set is used. The average number of physical pages accessed by our workload sets, as a percentage of total physical pages is 74% for HM, 42% for LM and 63% for MX, meaning physical memory is uniformly accessed across all vaults, which results in a fair comparison among the different page policies in our experiment. The simulations are fast-forwarded through their initialization phases and caches are warmed-up for an additional 100 million instructions. Then detailed simulations are run for 800 million instructions for each set of workloads.

## 4.3 Evaluated Schemes

In our experiments, we consider a wide variety of page-closure policies which are implemented both statically (S) and dynamically (D):

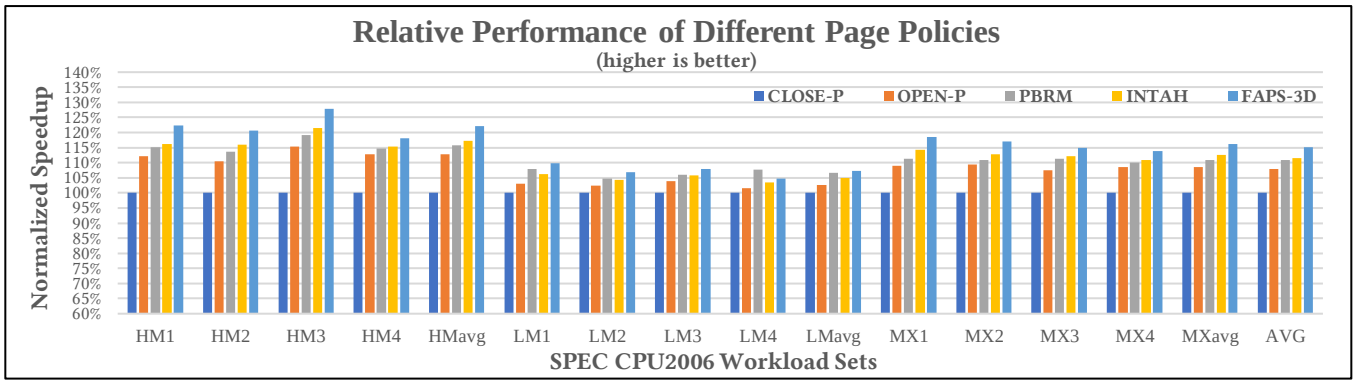



**Figure 6. Normalized Speedup of FAPS-3D over other schemes**

- **CLOSE-P (S):** the baseline *close-page* policy, where row-buffer is precharged if no pending request in the queue goes to this row, making room for next row to be activated upon request. Scheduling policy chosen for this scheme in FCFS. This scheme favors the applications with low spatial locality.
- **OPEN-P (S):** the *open-page* policy, where row-buffer is kept open after the current access in anticipation of upcoming request to be a hit in the open row. This scheme favors the applications with high spatial locality. Scheduling policy chosen for this scheme is FR-FCFS [5].
- **PBRM (D):** Prediction Based Row-buffer Management [13]. This is an implementation of *access-based* page policy where a history table is used to keep track of number of hits in recently opened rows before closing it. In this scheme, when a row is opened, a corresponding entry for that row is looked up in the table. If no entry is found, page is kept open until a conflict happen and the number of hits is recorded in the table before closing this row. If an entry is found, the row is closed after the estimated number of accesses as suggested by the table entry. The entries are updated periodically in the table based on the correctness of the prediction. This scheme maintains the record of a few recently accessed rows per bank and provides moderate history table hit rate in the random stream of memory access requests to make informative page closure decisions.
- **Intel Adaptive HAPPY (INTAH) (D):** HAPPY implementatation of Intel Adaptive page closure policy [28], [35]. This is an implementation of *time-based* page policy where a *timing register* (TR) and a *timeout counter* (TC) are used and a page is kept open until TC reaches TR and then closed afterwards. A scheme is used to update the value of TR at runtime using a 4-bit *mistake counter* (MC) whose value is incremented when a page miss happens that could have been a page hit and decremented when a page miss happens that could be a page conflict. If MC is higher than a threshold, page is kept open for a longer period, otherwise page is closed sooner. This scheme was originally implemented in Intel Xeon *X*5650 [28] and authors in [35] implemented the modified version of this scheme, which achieves similar performance results while incurring considerably lower storage overhead.
- **FAPS-3D (D):** Our feedback-directed, adaptive page management scheme, which is an implementation of dynamic *access-based* page policy as discussed in detail in section 3.

# 5. EXPERIMENTAL RESULTS

## 5.1 Performance Improvement

Figure 6 shows the performance improvement obtained by our scheme over the *CLOSE-P*, *OPEN-P*, *PBRM* and *INTAH* schemes mentioned in previous section. The performance of each scheme is calculated by taking the *geometric mean* of IPC (instructions-per-cycle) of each application in the workload set. The performance of each scheme is normalized to that of *CLOSE-P*. The average performance improvement of *FAPS-3D* for *high-memory-intensive* workloads (HMx) is 22.2% over *CLOSE-P*, 8.4% over *OPEN-P*, *5.6%* over PBRM, and 4.2% over *INTAH* scheme, respectively. The average performance improvement of *FAPS-3D* for *low-memory-intensive* workloads (LMx) is 7.3% over *CLOSE-P*, 4.5% over *OPEN-P*, 0.7% over *PBRM*, and 2.2% over *INTAH*, respectively. Finally, for *mixed-memory-intensive* workloads (MXx), our scheme achieves an average performance improvement of 16.1% over *CLOSE-P*, *6.9%* over *OPEN-P*, 4.6% over PBRM, and 3.2% over *INTAH*, respectively. Overall, our scheme gains the average performance improvement of 15.2% over *CLOSE-P*, 6.9% over *OPEN-P*, 4.0% over *PBRM*, and 3.3% over *INTAH* scheme, respectively, across all 12 sets of workloads.

As can be seen from Figure 6, the *high-memory-intensive* workloads gain higher performance improvement than their *low-memory-intensive* counterparts. This is because HMx workloads tend to reference main memory more often and will benefit more from the efficient management of memory access requests. The LMx workloads are less memory intensive as compared to HMx, their gains are also lower accordingly. The MXx workloads are mixes of HMx and LMx workloads, and their performance gains are moderate. Note that for LMx workloads, the performance of PBRM is close to that of FAPS-3D, but our scheme has a clear advantage over PBRM, as the storage overhead is 98x less. *PBRM* decides page closure policy at the granularity of DRAM page but due to limited record of DRAM pages in history table and increasingly random sequence of DRAM pages accessed in multi-programming environment,

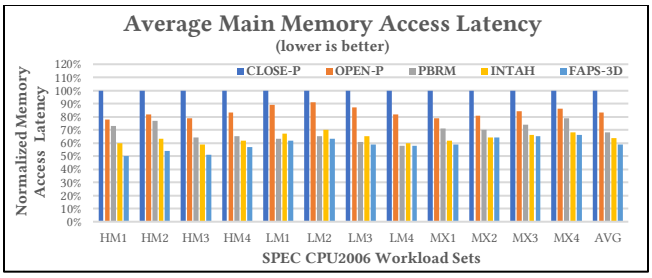



**Figure 7. Main Memory Access Latency**

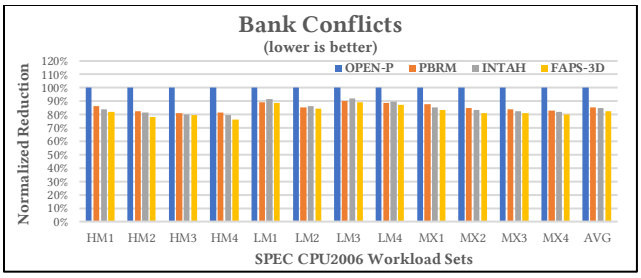



**Figure 8. Bank Conflicts**

the prediction accuracy is comparatively less, while our scheme *FAPS-3D*, despite being implemented on the bank-level granularity, performs better with much lower storage overhead as shows in the results. The HAPPY implementation of Intel Adaptive policy is based on the observation of a potential correlation between physical address bits generated by the processor and internal DRAM structure for a fixed address mapping scheme. Due to its global nature, their scheme does not consider dynamic and short-lived application's behavior changes, which is also admitted by the authors [35]. Our scheme proactively considers this dynamically changing behavior of application at run-time and adjusts the policy accordingly, resulting in overall performance improvement.

## 5.2 Main Memory Access Latency

The performance gains of our scheme are made possible by the reduction in overall *main memory access latency* (MMAL). Figure 7 shows this reduction achieved by our adaptive page management scheme. The results are normalized to the *MMAL* of *CLOSE-P*. The *OPEN-P* scheme reduces the *MMAL* by 16.6% over the *CLOSE-P*, *PBRM* achieves an average reduction of 31.7%, *INTAH* achieves an average reduction of 36.2%, and our scheme *FAPS-3D* reduces the MMAL by 41.0% on average when compared to *CLOSE-P*. Also note that *MMAL* in this case represents the latency to access only the 3D-stacked DRAM based main memory, which does not include access latency of caches. This latency includes the request/response delays, scheduling delays and delays associated with the bank unavailability due to refresh operations.

Our scheme focuses on improving the access time for the banks that have varying row-buffer hit-rates by adaptively changing the page mode for the banks. This results in considerable reduction in *MMAL* over static page policies like *CLOSE-P* and *OPEN-P* that are not dynamically adjusted during different application execution phases. A key takeaway here is the dynamic page policies in our experiments, including *PBRM*, *INTAH* and *FAPS-3D*, all perform better than static page policies like *CLOSE-P* and *OPEN-P*, which means page policies that adapt to the dynamically changing application's memory access behavior in many-core environment are more suitable than the static page policies and are ideal candidates to be implemented in high-performance systems. The delays associated with accessing the main memory is a major factor in determining the overall system performance and reducing *MMAL* translates directly to overall performance improvement, especially for high memory-intensive workloads. This is made possible by reduction in bank conflicts due to adaptive page and scheduling policy as discussed in section 3.

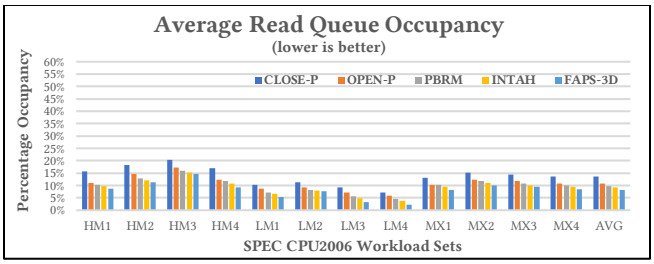
## 5.3 Bank Conflicts

Figure 8 shows average reduction in bank conflicts of different dynamic page policies (i.e. *PBRM*, *INTAH* and *FAPS-3D*) as compared to static *OPEN-P* policy. Bank conflicts of *OPEN-P* policy for various workload sets are normalized to 100% and other schemes are compared accordingly. For HMx workload sets, there is an average reduction of 17.2%, 18.6% and 21.0% for *PBRM*, *INTAH* and *FAPS-3D*, respectively, in bank conflicts. For LMx workload sets, the average reduction is 11.7%, 10.2% and 12.7% for *PBRM*, *INTAH* and *FAPS-3D*, respectively. Finally, for MXx workload sets, the average reduction achieved is 15.2%, 16.6%, and 18.6% for *PBRM*, *INTAH* and *FAPS-3D*, respectively. Overall, average reduction in bank conflicts is 14.7%, 15.1% and 17.5% for *PBRM*, *INTAH* and *FAPS-3D*, respectively, across all workloads.

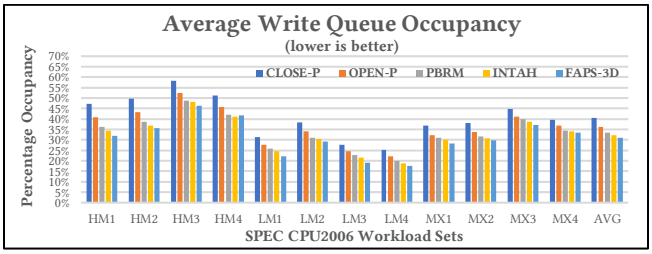
As applications differ widely in memory access behavior, implementing a static open-page policy will not yield optimal results in all circumstances. As can be seen from Figure 8, dynamic page policies achieve considerable reduction in overall bank conflicts since they all occasionally implement *close-page* policies when there is not enough locality within the access streams that can take advantage of *open-page* policy. HMx workload sets get the largest reduction in bank conflicts because of their memory intensive behavior. LMx workload sets on the other hand, get the lowest reduction due to lower access rates and lower bank conflicts originally, while MXx's reduction in bank conflicts lies between the two. Note that *PBRM* and *INTAH* are implementations of dynamic *access-based* and *time-based* page policy respectively, they yield similar results in achieving bank conflict reduction in our experiments. *INTAH* yields a minor improvement of 0.4% over *PBRM* in bank conflict reduction. FAPS-3D achieves a reduction of 2.4% and 2.8% over *INTAH* and *PABRM*, respectively. This reduction is achieved due to the close monitoring and independent management of each bank by the vault controller according to its specific access pattern in different execution phases.

## 5.4 Average Queue Occupancy

Given the availability of hundreds of banks spread across multiple vaults, the request queues are intrinsically shallower for 3D-stacked DRAM-based main memory system as compared to their traditional DDRx-based counterparts. Figure 9(a) and (b) shows average occupancy for read and write queues respectively as seen by an incoming request. For read queues, *CLOSE-P* policy




**(a)**




**(b)**

**Figure 9. Average (a) Read Queue Occupancy, (b) Write Queue Occupancy**

has an average occupancy of 13.7%, while *OPEN-P, PBRM, INTAH* and *FAP-3D* reduce the average read queue occupancy to 10.9%, 9.8%, 9.2% and 8.2%, respectively. Similarly, for write queues, *CLOSE-P* policy have an average occupancy of 40.6% while *OPEN-P, PBRM, INTAH* and *FAP-3D* reduce the average write queue occupancy to 36.2%, 33.5%, 32.4% and 31.0%, respectively. This reduction is achieved due to fewer bank conflicts as explained earlier, resulting in more efficient processing of requests and hence lower waiting time for each memory request to get processed. Note that the write queues have higher average occupancy as compared to read queues because priority is given to read requests due to their critical nature in causing potential processor stalls.

## 5.5 Energy Consumption

Energy consumption of 3D-stacked DRAM across all vaults is shown in Figure 10. The average energy consumption of *CLOSE-P* is normalized to 100% and other schemes are compared accordingly. Our scheme *FAPS-3D* reduces the overall memory energy consumption by 12.7% on average compared to *CLOSE-P*, 5.7% compared to *OPEN-P*, 3.4% compared to *PBRM*, and 2.2% compared to *INTAH*, respectively. The detailed energy model used in our simulation is based on [32], [33] and incorporates energy consumed by activation/precharge operations, read and write operations, refresh operations and energy associated with dynamically managing *power-down* mode when no references are due.

The reduction in overall energy consumption is due to accumulated savings from fewer activation and precharge operations as compared to *CLOSE-P*, which implements *close-page* policy and may have to activate the same row it just precharged. *OPEN-P* policy accounts for less energy consumption as it minimizes the frequency of activate and precharge operations due to locality found within applications resulting in repeated accesses

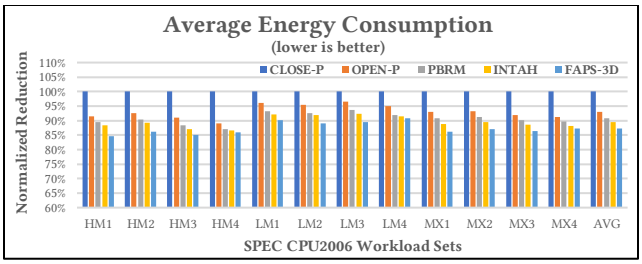



**Figure 10. Average 3D-DRAM Energy Consumption**

to the row open in the row-buffer before closing it. *PBRM* and *INTAH* consumes 2.3% and 3.5% less energy respectively as compared to *OPEN-P* policy due to better prediction accuracy of page closing decisions. Our scheme *FAPS-3D* saves additional energy by closing the banks adaptively when applications are in their execution phases with low locality and by putting the banks in low-power mode (incurring just refresh operations) when no accesses are due for an extended time duration.

## 6. RELATED WORK

**Dynamic Page Policies:** Various dynamic page policies have been studied over the years for DDRx-based main memory systems in pursuit of minimizing the bank conflicts and improving memory access latency and hence, overall system performance. As mentioned earlier, dynamic page policies are categorized into *access-based* policies and *time-based* policies. One *access-based* policy is introduced by Huan et al. [29] called the processor-directed page policy where processor keeps track of page hits and misses, and sends this information to memory controller to accurately make future page closure decisions. Xu et al. [25] proposed a two-level dynamic page policy where a 2-bit saturating counter is kept for each of the last *n* DRAM-page accesses and predict the future page closure for those DRAM pages. Stankovic et al. [34] proposed a *time-based* page policy where the concept of *live-time* (interval between opening a row until last access to that row) and *dead-time* (interval between last access to an open row and before closing it) is used to predict the time of page closure. Kaseridis et al. [30] proposed to leave the page open speculatively for $t_{RC}$ period which is the minimum time between the two activation in the same bank. All the above-mentioned schemes are based on traditional DDRx-based main memory systems while our scheme is optimized for 3D-stacked DRAM based main memory system that provides immense memory level parallelism and independent control over the set of banks in the vault.

**Other Memory Controller Improvements:** Various memory controller improvement schemes including scheduling policies and address mapping policies have been studied over the years to improve the performance of main memory system. Rixner et. al. [5] introduced the first-ready first-come first-serve memory scheduling policy (FR-FCFS) that focuses on the reordering of the memory accesses while exploiting the data locality found within applications to maximize the row-buffer hits. Mutlu and Moscibroda [17], [23] proposed memory access scheduling policies, optimizing fairness in the memory latency and quality of service (QoS) for multi-threaded applications running concurrently in chip multiprocessor systems. Kim et. al. [16],

[22] dig deeper into the memory access behavior of different threads and quantify the threads with similar behavior and schedule them adaptively with aim to find the optimal balance between fairness and overall system throughput. Carter et. al. [12] proposed a modified memory controller that includes the combination of a configurable application-specific physical address remapping and memory prefetching to enhance the performance of memory intensive applications. Several other studies [6], [9], [15], [19], [24], [27], [31] have proposed different optimizations in designing efficient memory controllers with improvements in different domains ranging from data reorganization for exploiting locality, memory prefetching and power efficiency. Our adaptive page management scheme works orthogonal to these approaches and can be applied in conjunction with these policies to further enhance the performance of memory controller.

## 7. CONCLUSION

In this paper we proposed *FAPS-3D*, a feedback directed adaptive page management scheme for 3D-stacked DRAM based main memory system. This scheme is designed for many-core systems where multiple applications having different memory access behaviors are running concurrently and impose varying demands of memory bandwidth during different execution phases. Our scheme incorporates a policy engine that resides in the vault controller of the logic base of 3D-stacked DRAM, and monitors the behavior of each bank based on the row-buffer hit-rate and identifies banks with different hit-rates as the candidate of *open-page* or *close-page* mode accordingly. It also adjusts the request scheduling that works with corresponding page mode in order to minimize the overall memory access latency. Our adaptive scheme results in reasonable reduction in main memory access latency, which leads to an average performance improvement of 15.2% over the base scheme of *close-page* mode. Our scheme also performed 4.0% better than an existing *access-based* page management policy while incurring 98x less storage overhead. Our scheme also outperformed a state-of-the-art, *time-based* HAPPY implementation of Intel Adaptive age policy by 3.3% over a wide variety of SPEC CPU2006 benchmarks of varying memory intensity. Our scheme can be implemented with minimum design complexity in the logic base of the 3D-stacked DRAM. This scheme works completely in the vault controller and does not need any modification to main memory controller residing on the processor, making it feasible for wide variety of applications and platforms.

## ACKNOWLEDGEMENT

We thank the anonymous reviewers for their valuable feedback. This work is supported in part by the National Science Foundation under grant CCF-1513899. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors only and do not necessarily reflect the views of the National Science Foundation.

## REFERENCES

[1] W.A. Wulf and S.A. McKee, Hitting the Memory Wall: Implications of the obvious, ACM SIGARCH Computer Arch. News, March 1995, pages 20-24.
[2] Hybrid Memory Cube Specification 2.1. HMC Consortium, November 2015.
[3] Micron. 8Gb DDR3 SDRAM: MT41K1G8 https://www.micron.com/-/media/client/global/documents/products/datasheet/dram/ddr3/8gb_ddr3l.pdf
[4] G. Kim, Jo. Kim, J. H. Ahn and J. Ki. Memory-centric system interconnect design with hybrid memry cubes. In PACT, 2013.
[5] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens. Memory access scheduling. In ISCA-27, 2000.
[6] E. Ipek, O. Mutlu, J. F. Martinez, and R. Caruana. Self-Optimizing memory controllers: A reinforcement learning approach. In ISCA-35, 2008.
[7] J. Ahn, S. Yoo, and K. Choi, "Low-power hybrid memory cubes with link power management and two-level prefetching" IEEE Trans. on VLSI Systems, 4(2), 2016.
[8] D. H. Woo, N. H. Seong, D. L. Lewis and H. S. Lee. An optimized 3-D stacked memory architecture by exploiting excessive, high-density TSV bandwidth. In HPCA-16, 2010, pages 1-12.
[9] B. Akin, F. Franchetti, J. C. Hoe. Data reorganization in memory using 3D-Stacked DRAM. In ISCA-42, 2015, pages 131-143.
[10] A. Hansson, N. Agarwal, A. Kolli, T. Wenisch, A. N. Udipi. Simulating DRAM controllers for future system architecture exploration. ISPASS 2014, pages 201-210.
[11] J. T. Pawlowski, Hybrid Memory Cube (HMC). In HotChips-23, 2011.
[12] J. Carter, W. Hsieh, L. Stoller, M. Swanson, L. Zhang, E. Brunvand, A. Davis, C. C. Kuo, R. Kuramkote, M. Parker, L. Schaelicke and T. Tateyama. Impulse: Building a smarter memory controller. In HPCA-5, 1999, page 70.
[13] M. Awasthi, D. W. Nellans, R. Balasubramonian, Al Davis. Prediction Based DRAM Row-Buffer Management in the Many-Core Era. In PACT, 2011, pages 183-184.
[14] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood. The gem5 simulator. May 2011, ACM SIGARCH Computer Architecture News. http://www.gem5.org.
[15] J. Ahn, S. Yoo, O. Mutlu, K. Choi, PIM-Enabled Instructions: A low-overhead, locality-aware processing-in-memory architecture. In ISCA, 2015, pages 336-348.
[16] Y. Kim. D. Han, O. Mutlu, and M. Harchol-Balter. ATLAS: A scalable and high-performance scheduling algorithm for multiple memory controllers. In HPCA-16, 2010, pages 1-12.
[17] O. Mutlu and T. Moscibroda. Stall-time fair memory access scheduling for chip multiprocessors. In Micro-40, 2007, pages 146-160.
[18] E. Azarkhish, D. Rossi and I. L. Dei. High performance AXI-4.0 based interconnect for extensible smart memory cube. In DATE, 2015, pages 1317-1322.
[19] M. Rafique and Z. Zhu. CAMPS: Conflict-Aware Memory-Side Prefetching Scheme for Hybrid Memory Cube. In ICPP-47, 2018.
[20] J. L. Henning, "SPEC CPU2006 benchmark descriptions," ACM SIGARCH Computer. Archit. News, 34(4), pages 1-17, 2006.
[21] Standard Performance and Evaluation Corporation. https://www.spec.org/
[22] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter. Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior. In MICRO-43, 2010, pages 65-76.
[23] O. Mutlu and T. Moscibroda. Parallelism-aware batch-scheduling: Enhancing both performance and fairness of shared DRAM-systems. In ISCA-35, 2008, pages 63-74.
[24] I. Hur and C. Lin. Adaptive history-based memory schedulers. In MICRO-37, 2004, pages 343-354.
[25] Y. Xu, A.S. Agarwal, B.T. Davis. Prediction in dynamic sdram controller policies. In SAMOS, 2009, pages 128-138.
[26] Hybrid Memory Cube Consortium. http://hybridmemorycube.org/
[27] C. J. Lee, O. Mutlu, V. Narasiman, and Y. N. Patt. Prefetch-aware DRAM controllers. In MICRO-41, 2008, pages 200-209.
[28] Intel Xeon Processor x5650.
[29] D. Huan, Z. Li, W. Hu, Z. Liu. Processor directed dynamic page policy. In ACSAC, 2006, pages 109-122.
[30] D. Kaseridis, J. Steucheli, and L. K. John. Minamilist open-page: A dram page-mode scheduling policy for many-core era. In MICRO, 2011, pages 24-35.
[31] Z. Zhang, Z. Zhu, and X. Zhang. A permutation based interleaving scheme to reduce row-buffer conflicts and exploit data locality. In MICRO-33, 2000, pages 32-41.
[32] K. Chandrasekar, C. Weis, Y. Li, S. Goossens, M. Jung, O. Naji, B. Akesson, N. Wehn, K. Goossens. DRAMPower: Open-source DRAM Power & Energy Estimation Tool.
[33] M. Jung, C. Weis, N. Wehn, M. Sadri and L. Benini. Optimized active and power-down mode refresh control in 3D-DRAMs. In VLSI-SoC-22, 2014.
[34] V. V. Stankovic, N.Z. Milenkovic. DRAM controller with a close-page predictor. In EUROCON, 2005, pages 693-696.
[35] M. Ghasempour, A. Jaleel, J. D. Garside, M. Lujan. HAPPY: Hybrid Address-based Page Policy in DRAMs. In Memsys, 2016, pages 311-321.
[36] High-Bandwidth Memory https://www.amd.com/system/files/documents/high-bandwidth-memory-hbm.pdf
[37] Muralimanohar, R. Balasubramonian, N. Jouppi. Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0. In MICRO, 2007, pages 3-14.